

Python Visualization Library Documentation

Internship Task – Visualization Library Documentation

1. Library Overview

Matplotlib

Matplotlib is a foundational Python plotting library used to create static, highly customizable figures. It is widely used in data analysis, scientific computing, engineering, academic work, and reporting.

Key features: detailed control over axes, labels, colors, markers, legends, annotations, layouts, and figure size.

Seaborn

Seaborn is a high-level statistical visualization library built on Matplotlib. It works particularly well with Pandas DataFrames and provides attractive defaults with concise syntax.

Key features: statistical graphics, distributions, categorical comparisons, relationship analysis, themes, and grouping with variables such as hue.

2. Installation and Basic Setup

```
pip install matplotlib seaborn pandas
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
```

3. Matplotlib Graph Types

Line Plot

Description / Use Case: Shows trends over an ordered variable such as time.

```
x=[1,2,3,4,5]\ny=[10,15,13,20,25]\nplt.plot(x,y,marker="o")\nplt.xlabel("Day")\nplt.ylabel("Sales")\nplt.title("Sales Trend")\nplt.show()
```

Scatter Plot

Description / Use Case: Shows the relationship between two numerical variables.

```
x=[1,2,3,4,5]\ny=[2,5,4,8,10]\nplt.scatter(x,y)\nplt.xlabel("Study Hours")\nplt.ylabel("Marks")\nplt.title("Study Hours vs Marks")\nplt.show()
```

Bar Chart

Description / Use Case: Compares values across discrete categories.

```
categories=["A","B","C","D"]\nvalues=[25,40,30,50]\nplt.bar(categories,values)\nplt.xlabel("Category")\nplt.ylabel("Value")\nplt.title("Category Comparison")\nplt.show()
```

Histogram

Description / Use Case: Shows the frequency distribution of numerical values.

```
marks=[45,50,52,55,60,61,65,70,72,75,80,85]\nplt.hist(marks,bins=5,edgecolor="black")\nplt.xlabel("Marks")\nplt.ylabel("Frequency")\nplt.title("Marks Distribution")\nplt.show()
```

Pie Chart

Description / Use Case: Shows how a total is divided among a small number of categories.

```
labels=["Python","Java","C","Other"]\nvalues=[40,25,20,15]\nplt.pie(values,labels=labels,autopct="%1.1f%%")\nplt.title("Language Preference")\nplt.show()
```

Box Plot

Description / Use Case: Summarizes median, quartiles, spread, and possible outliers.

```
data=[12,15,14,18,21,22,24,30,35]\nplt.boxplot(data)\nplt.ylabel("Value")\nplt.title("Box Plot")\nplt.show()
```

Area Plot

Description / Use Case: Emphasizes magnitude using a filled area.

```
x=[1,2,3,4,5]\ny=[10,12,18,15,22]\nplt.fill_between(x,y,alpha=0.5)\nplt.plot(x,y)\nplt.title("Area Plot")\nplt.show()
```

Subplots

Description / Use Case: Places multiple related charts in one figure.

```
fig,ax=plt.subplots(1,2,figsize=(8,3))\nax[0].plot([1,2,3],[2,4,3])\nax[0].set_title("Line")\nax[1].bar(["A","B","C"],[3,5,4])\nax[1].set_title("Bar")\nplt.tight_layout()\nplt.show()
```

4. Seaborn Graph Types

Line Plot

Description / Use Case: Shows trends and supports grouping with semantic variables.

```
df=pd.DataFrame({"Day":[1,2,3,4,5],"Sales":[10,15,13,20,25]})\nsns.lineplot(data=df,x="Day",y="Sales",marker="o")\nplt.title("Sales Trend")\nplt.show()
```

Scatter Plot

Description / Use Case: Explores relationships and can use hue to distinguish groups.

```
df=pd.DataFrame({"Hours":[1,2,3,4,5],"Marks":[45,50,65,72,85],"Group":["A","A","B","B","B"]})\nsns.scatterplot(data=df,x="Hours",y="Marks",hue="Group")\nplt.title("Study Hours vs Marks")\nplt.show()
```

Bar Plot

Description / Use Case: Compares numerical values across categories.

```
df=pd.DataFrame({"Department":["CSE","ECE","EEE","MECH"],"Marks":[82,76,71,68]})\nsns.barplo\n t(data=df,x="Department",y="Marks")\nplt.title("Marks by Department")\nplt.show()
```

Histogram / Distribution

Description / Use Case: Shows the shape and spread of numerical data.

```
data=[45,50,52,55,60,61,65,70,72,75,80,85]\nsns.histplot(data,bins=5,kde=True)\nplt.title("M\narks Distribution")\nplt.show()
```

Box Plot

Description / Use Case: Compares distributions across categories and highlights possible outliers.

```
df=pd.DataFrame({"Department":["CSE","CSE","CSE","ECE","ECE","ECE"],"Marks":[70,82,90,65,75,\n80]})\nsns.boxplot(data=df,x="Department",y="Marks")\nplt.title("Marks by\nDepartment")\nplt.show()
```

Violin Plot

Description / Use Case: Shows distribution shape using a density-based violin form.

```
df=pd.DataFrame({"Group":["A","A","A","B","B","B"],"Score":[60,65,70,72,80,85]})\nsns.violin\n plot(data=df,x="Group",y="Score")\nplt.title("Score Distribution")\nplt.show()
```

Count Plot

Description / Use Case: Counts observations in each category.

```
df=pd.DataFrame({"Language":["Python","Python","Java","C","Python","Java"]})\nsns.countplot(\ndata=df,x="Language")\nplt.title("Language Counts")\nplt.show()
```

Heatmap

Description / Use Case: Uses color intensity to represent values in a matrix.

```
data=pd.DataFrame({"Math":[80,70,90],"Science":[75,85,88],"English":[70,80,82]})\nsns.heatma\n p(data corr(),annot=True,cmap="coolwarm")\nplt.title("Correlation Heatmap")\nplt.show()
```

5. Customization Examples

Matplotlib

```
plt.plot(x,y,color="green",marker="o",linestyle="--",linewidth=2)\nplt.xlabel("X Value")\nplt.ylabel("Y Value")\nplt.title("Customized Line Plot")\nplt.grid(True)\nplt.show()
```

Seaborn

```
sns.set_theme(style="whitegrid")\nsns.scatterplot(data=df,x="Hours",y="Marks",hue="Group",s=100)\nplt.title("Styled Scatter Plot")\nplt.show()
```

6. Comparison: Matplotlib vs Seaborn

Aspect	Matplotlib	Seaborn
Ease of use	Flexible but can require more code.	High-level and concise for statistical charts.
Customization	Very extensive, fine-grained control.	Strong; uses Matplotlib underneath.
Interactivity	Primarily static by default.	Primarily static; built on Matplotlib.
Statistical plots	Possible, but often more manual.	Strong support for distributions and relationships.
DataFrame support	Good; often explicit x/y handling.	Excellent; designed around tidy DataFrames.
Large datasets	Many graphical elements can become slow.	Similar rendering considerations; aggregation/sampling can help.
Best use	Custom scientific/report-quality figures.	Fast exploratory and statistical visualization.

7. Practical Selection Guide

- Matplotlib: choose it when precise control over the complete figure is important.
- Seaborn: choose it for quick, attractive statistical and exploratory charts.
- Both: use Seaborn for convenient statistical plotting and Matplotlib for detailed formatting.
- Large datasets: consider aggregation, sampling, or specialized visualization approaches when drawing every raw observation is inefficient.

8. Beginner Workflow

```
df=pd.read_csv("data.csv")\nprint(df.head())\nprint(df.describe())\nsns.scatterplot(data=df,x="feature_1",y="feature_2")\nplt.xlabel("Feature 1")\nplt.ylabel("Feature 2")\nplt.title("Feature Relationship")\nplt.show()
```

9. Conclusion

Matplotlib and Seaborn are complementary Python visualization libraries. Matplotlib provides a flexible foundation for highly customized charts, while Seaborn provides a simpler interface for statistical and exploratory visualization. Learning both gives beginners a strong foundation for Python data analysis.

10. Resources

- Matplotlib Quick Start: https://matplotlib.org/stable/users/explain/quick_start.html
- Seaborn Introduction: <https://seaborn.pydata.org/tutorial/introduction.html>
- Matplotlib Documentation: <https://matplotlib.org/stable/>
- Seaborn Documentation: <https://seaborn.pydata.org/>